使用者回來時，停放的 KV 該怎麼送回 GPU？H200 上事先登記的組合地圖

When the Session Returns: A Preregistered Decision Map for Restoring Parked KV Cache on H200（英文暫定題）

> v4.2（2026-10-30）。標記：〔實測〕＝在 H200 上量的；〔模擬〕＝trace 模擬；〔判讀〕＝作者的解讀。方括號是 `EVIDENCE.md` 的列號。〔待決〕＝寫法等「待你決定」。

> **待你決定**（內部用；數字都已核對，內文照「建議」寫，受影響的句子標〔待決〕）
>
> 1. **投稿類型**：【已定】（使用者 10/30）：ICPE 2027 研究 track 的 **EERCS**（Empirical／Experience／Reproduction／Case Study）入口。
> 2. **AI 揭露**（§3.5）：「作者人工檢查了 oracle、模擬器、量化與計時程式」這句，要你確認真的做過才留。
> 3. **頁數**：本稿 9 張圖。英文版超過 10 頁時，建議依序把圖 4（模擬驗證）縮成表 1 的一列、圖 6（FreeInference 容量）移到 artifact、圖 3（方法流程）併進圖 2。
> 4. **「最好」的品質規則**（§4）：本稿用「主測驗（32K、192 題）過 ε 才算」，對 INT4 與低秩一視同仁；Qwen 的 INT4 在多鍵 NIAH 的 Cake＋GPU 忙讀法沒過，用 † 標出。另一個選法是「每種讀法都要過」，那樣 Qwen 慢層忙格的最好會改成 INT4 全部載入。
> 5. **FreeInference 那句的範圍**（§7.2）：建議「前 6 週、四種回來時上界、測過的 CPU 16–1,024 GiB 都 <15%；知道誰不回來只在另一週、128–1,024 GiB 量過」。
> 6. **打字提示的數字**（§7.4）：建議主引 GPU 縮小重播的 0.81／0.79，全尺寸模擬的中位數當輔助，並限定「只對預取」。
> 7. **知道誰不回來的條件**（§7.3）：建議「11 格裡 10 格在 CPU ≤256 GiB、全部在 b32／b48」。
> 8. **MPS 圖**（§6）：圖 5 同時畫 H200 常數版（藍實線）與登記版（灰虛線）；建議只留 H200 常數版。

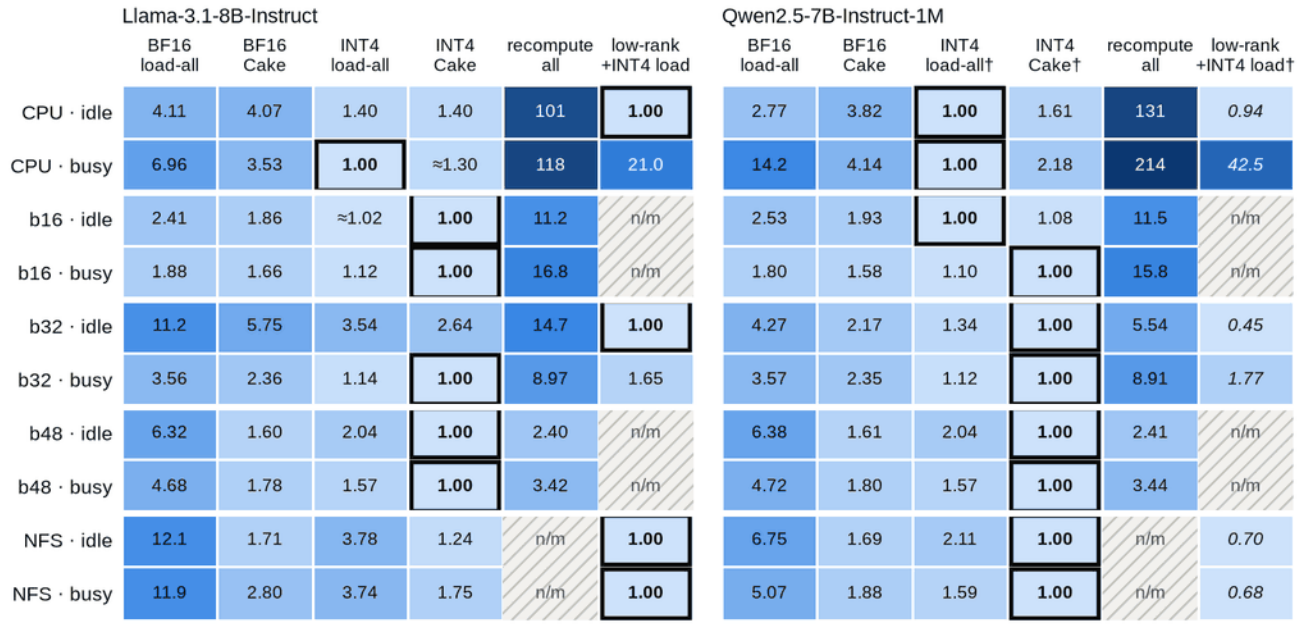
名詞表

名詞	意思
停放的 KV、回來請求	對話在兩次請求之間移出 GPU 的鍵值快取（key-value cache，KV cache；下稱 KV）；同一段對話的下一則訊息叫回來請求，它要先把停放的 KV 送回 GPU
TTFT	首字延遲（time to first token）：從送出下一則訊息，到看到回答第一個字的時間
CPU 層、慢層	停放 KV 的地方。CPU 層是主機記憶體；慢層是限速模擬的層，不是真的裝置
SSD 級、NFS 級	用頻寬上限模仿固態硬碟（solid-state drive，SSD）與網路檔案系統（Network File System，NFS）的慢層
b16／b32／b48	把頻寬限到剛好讓 Cake 在第 16／32／48 塊會合（32K 共 64 塊）的慢層，所以頻寬依模型不同
全部載入、全部重算、Cake	三種送回的方式：把存的 KV 整段搬回；在 GPU 上從頭重算；GPU 從第一塊往後重算、同時從最後一塊往前載入，在中間會合
BF16、INT4、低秩＋INT4	三種 KV 格式：bfloat16（BF16，16 位元浮點數，原精度）；4 位元整數（4-bit integer，INT4），位元組小 3.2 倍；先做低秩分解（例如奇異值分解，singular value decomposition，SVD）再量化，位元組小約 8–17 倍
一次拆完、逐塊解壓	INT4 送回 GPU 後轉回可用格式的兩種實作：整段讀完一次轉；每讀一塊就轉一次
GPU 閒、GPU 忙	還原時 GPU 上有沒有其他工作；地圖的「忙」是合成的固定背景負載
stream 共享、MPS、SM	讓送回與別的工作同時用一張 GPU 的兩種方式。stream 共享：兩邊的工作各放在一條 CUDA stream（GPU 上的工作佇列）、不設上限，由 GPU 交錯執行。MPS（NVIDIA 的多行程服務，Multi-Process Service）：讓多個程式同時在同一張 GPU 上執行，並能限制每個程式最多使用多少 SM（streaming multiprocessor，GPU 上的計算單元）；本文限制別的工作最多用 87%、75% 或 50%，或不設上限
領先、翻轉	領先＝1 − 最好的時間 ÷ 第二好的時間。翻轉＝條件改變後，最快的做法換成另一個；背景實驗登記的門檻是至少 2 條線換了、原本最快的做法慢 ≥ max(10%, δ*)，且 95% 信賴區間（confidence interval，CI）的下界 >0 [M3-b]；δ* 是時間門檻（§3.1）
截尾、寫入飽和	截尾：離 trace 結尾太近的「不再回來」判斷不算；寫入飽和：慢層寫不完，模擬讓還沒寫完的資料就能讀，所以偏樂觀

名詞	意思
oracle、上界	oracle（完美預測器）和對手做同樣的事，只多知道未來的一件事；它比最好的不預知對手少花的 TTFT 比例叫上界
四種回來時上界	預取、淘汰順序、回合結束就丟、輪到淘汰時才丟；「知道誰不回來」另外列
打字提示預取	從使用者開始打字起就預取；不需要預測模型，但要客戶端送出打字事件；提前量用打字速度模型算
四個方向	圖 2(b) 的整理軸：重用（怎麼送回、近似重算）、壓縮（格式）、聚焦（只讀重要的部分）、擴展（多一層、多一條路、預測）；GPU 資源跨各方向

一頁看懂

1. **場景**：使用者和聊天機器人或 agent 對話，兩則訊息之間常隔幾秒到幾分鐘。這段時間，服務端把這段對話的鍵值快取（key-value cache，KV cache；模型為了不重算前文而留下的中間結果）移出 GPU，放到 CPU 記憶體或更慢的儲存。使用者再發問時，KV 要先回到 GPU，畫面才會出現第一個字。一段 32K token 的 Llama 對話，KV 有 4.0 GiB [M1]。
2. **三個問題，一個接一個**：第一，不加任何預測時，KV 該怎麼送回、存成什麼格式？答案會不會因情況改變？第二，GPU 同時在跑別的工作時，要保留多少 GPU 給送回？第三，在前兩題的固定做法之上，再加預測或更複雜的機制，還能省多少？
3. **方法**：一張 NVIDIA H200 GPU、兩個 7–8B 模型、三份真實的使用紀錄（trace）；「值得做」的門檻在看到數字前就登記。送回方式與格式直接在 GPU 上量（32K token 的一次送回）。GPU 共用與預測用模擬計算，再在 GPU 上跑一個縮小版檢查：不算排隊時間時，實測÷模擬＝0.911–0.987，模擬給的排名和實測相同 [B6]。
4. **該怎麼送回**：放在慢速儲存層時（本文的 b32、b48 與 NFS 級：b32、b48 是把頻寬限到讓 Cake 在 64 塊的第 32、48 塊會合的層，Llama 為 2.49 與 0.722 GiB/s；NFS 級模擬網路檔案系統〔Network File System，NFS〕，0.331 GiB/s），4 位元整數格式（4-bit integer，INT4）加 Cake（ICML 2025 提出的做法：GPU 重算前段、同時載入後段）最快，比次好的做法快 11.0–40.7% [M1]。放在 CPU 記憶體時，關鍵是 4 位元的資料怎麼解壓：整段讀完再一次解壓，整段載入最快；每讀一塊就解壓一次（GPU 忙時），Cake 比較快。同一種做法只因解壓方式不同，就差到約 6.8 倍 [M1、M3-g、E3]。整段重算在 32K 時從來不是最好 [M1]。
5. **答案穩定**：在測過的條件下，不論重算變快，或 GPU 同時跑著像真實服務的背景工作，最快的做法都沒換；所以事先查一張表就夠 [A5、M3]。
6. **GPU 共用**〔模擬〕：GPU 同時在跑別的工作時，可以改用 NVIDIA 的多行程服務（Multi-Process Service，MPS）：它讓多個程式同時在同一張 GPU 上執行，並能限制每個程式最多使用多少計算單元（streaming multiprocessor，SM）。比起單純共用（兩邊的工作各放在一條 CUDA stream、不設上限，由 GPU 交錯執行），只開 MPS、先不限制別的工作，第一個字出現前的等待就縮短 30.7–62.9%，別的工作的處理量最多只少 6.5 個百分點。若再限制別的工作最多用 87%、75% 或 50% 的 SM，等於替送回保留更多 GPU，回答會更快，但別的工作會更慢；要保留多少，看各自更在乎哪一邊 [S2、M4]。
7. **再加預測**〔模擬〕：在真實的 agent 服務紀錄 FreeInference（前 6 週）上，就算預測完全準確，在測過的每種 CPU 記憶體大小都省不到 15% [B3-X1、B3-T5]。在另一組公開紀錄 Mooncake（FAST 2025）上，192 種組合裡有 36 種能省 15% 以上；這已扣掉「紀錄太短」造成的假象，也不算慢層寫不完的情況 [B9]。其中「先把資料搬回來」這一項，用「使用者開始打字」這個免費訊號，就能拿到完美預測約八成的好處 [C2-GPU]〔待決〕。
8. **其他機制**：漸進還原（先送最重要的部分）、近似重算、借用別的工作的算力，都沒過登記的門檻；借一張閒著的鄰卡多開一條搬運路徑過了，兩張卡時快 1.99 倍，但只量了 2 張卡 [G-A0、G-M12、G-M08、G-M06]。



Cell = restore time ÷ best in row (32K context, H200 measured). Bold + box = best; ≈ = weak win: (t − t_best) / t ≤ max(5%, row noise); n/m = not measured.
Low-rank column: fastest variant that passes ε (Llama: per-item SVD). Where no variant passes, the fastest is shown in italics and is not used as the best.
† fails ε (CI lower bound ≥ −2 pp) in at least one reading: Qwen INT4 (NIAH-T1 B-busy, B-i2b); Qwen low-rank (all variants).
Restore time only: excludes the write-time SVD that low-rank pays once per 32K request (Llama k19v29 3.24 s idle / 6.21 s busy).
INT4 load-all is the per-block dequant path on b16 / b48 and the one-shot unpack path on CPU / b32 / NFS. Throttle bandwidths per model: sidecar.

圖 1　組合地圖：每列的最快組合隨層、GPU 忙閒、模型與 INT4 的解壓實作變。〔實測〕32K 單次還原的時間 ÷ 該列最快，粗框是最快。格式要在主測驗過品質門檻 ε（答對率對 bfloat16〔BF16〕的 95% 信賴區間下界 ≥ −2 個百分點）才算最快；低秩不含寫入時做奇異值分解（singular value decomposition，SVD）的成本；† 是有一種讀法沒過 ε；b16、b48 兩列（會合點在第 16、48 塊的限速層）用逐塊解壓，其他列用一次拆完；模擬固態硬碟（solid-state drive，SSD）的 SSD 級沒有量 [M1、M2]。

摘要

多輪對話與 AI agent 的使用者，常隔一段時間才送出下一句話。這段期間，系統會把這段對話已經算好的中間資料，也就是鍵值快取（key-value cache，簡稱 KV cache），移出 GPU，放到 CPU 記憶體或更慢的儲存。下一句話來時，要先把它搬回 GPU，回答的第一個字才會出現；這段等待稱為首字延遲（time to first token，TTFT）。

搬回的做法有很多種：整段載入、整段重算，或兩者同時做（Cake，ICML 2025）；資料可以壓成 4 位元整數（4-bit integer，INT4）再存。GPU 被別的工作共用時，還能用 NVIDIA 的多行程服務（Multi-Process Service，MPS）讓兩邊同時執行，並限制各自能用多少 GPU 計算資源。此外也可以再加上預測，例如先猜誰會回來。本文在一張 NVIDIA H200 GPU 上，用兩個開源模型（Llama-3.1-8B、Qwen2.5-7B）實際量測這些做法，回答一個問題：什麼情況該用哪一種組合？

我們得到三個結論。

（一）照一張固定的表選就夠。 資料放在慢速儲存時，INT4 加 Cake 最快，比次佳做法快 11.0–40.7%。資料放在 CPU 記憶體時，INT4 怎麼解壓影響最大，差距可達約 6.8 倍。在測過的條件下，最佳做法不會因 GPU 忙不忙而改變。

（二）GPU 共用時，MPS 划算。 改用 MPS 讓搬回和別的工作同時執行，首字延遲縮短 31–63%，別的工作的處理量最多只少 6.5 個百分點（模擬）。

（三）再加預測，多半不划算。 在真實 agent 服務的使用紀錄（FreeInference）上，即使預測完全準確，也省不到 15%。在另一組使用紀錄（Mooncake）上，約五分之一的情況能省 15% 以上；其中「預先搬回」的好處，用「使用者開始打字」這個免費訊號就能拿到約八成。更複雜的機制裡，只有借用閒置的另一張 GPU 多開一條搬運路徑值得做（快 1.99 倍）。

給系統設計者的建議：先照表選格式與搬法；要不要建預測器，先在自己的使用紀錄上量出完美預測的上限，再決定。

（數字出處：[M1]、[M3-g]、[M3-b]、[S2]、[M4]、[B3-X1]、[B3-T5]、[B9]、[C2-GPU]、[G-M06]、[§0]）

1 引言：先畫地圖，再問固定的表之上還缺什麼

停放的鍵值快取（key-value cache，KV cache；下稱 KV）該怎麼送回，可以寫成一張表：只要知道 KV 停在哪一層、GPU 忙不忙、用哪個模型、4 位元整數格式（4-bit integer，INT4）用哪種解壓實作，就查得到答案。GPU 要和別的工作共用時，改用 NVIDIA 的多行程服務（Multi-Process Service，MPS；§6 說明）能讓回答快很多，但替送回保留得越多，別的工作就越慢。在這張表之上，預測只在部分使用紀錄（trace）上值得加 [M1、M3、M4、B3-T5、B9]。

聊天與 agent 讓同一段對話在幾秒到幾分鐘後又回來。GPU 記憶體要留給正在生成的請求，所以服務系統把等待中的對話的 KV 移到 CPU 記憶體或更慢的儲存（§11）。使用者回來時，系統要先把 KV 送回 GPU，才能開始算新訊息；這段等待使用者看得到，算在首字延遲（time to first token，TTFT：從送出訊息到看到回答第一個字的時間）裡。

業界從幾個方向縮短這段等待：改送回的方式、把 KV 壓小、只讀重要的部分、多一條路或提前開始，還能調整 GPU 分給送回的比例（§11）。每個方向都有系統報告收益，但比較的對象各不相同，而且大多一次只看一個方向。設計者要的是另一件事：手上的條件該選哪個組合，再加什麼才值得。

本文回答三個問題。第一，不加預測時，送回方式（全部載入、全部重算，或 ICML 2025 提出的 Cake：GPU 重算前段、同時載入後段）與格式（bfloat16〔BF16，16 位元浮點數〕、INT4、低秩分解加 INT4）怎麼選，這個選擇會不會隨情況改變（§4、§5）。第二，GPU 要和別的工作共用時，要保留多少 GPU 給送回；回答因此快多少，別的工作因此慢多少（§6）。第三，在固定的做法之上，再加預測（寫入時、回來時）或更複雜的機制（漸進、近似重算、借算力、多一條路徑），還能省多少，能不能用免費的訊號代替預測（§7、§8）。第二題在第一題的表上加 GPU 資源這個條件。第三題建立在第一題的發現上：§5 顯示一張固定的表就夠，所以第三題問的是這張表之上還缺什麼。

貢獻：

1. **組合地圖與穩定性**：在兩個模型、六種存放層、GPU 閒與忙的每一種情況下，量出哪個組合最快、比第二名快多少，全部在 GPU 上量；CPU 層的答案取決於 INT4 的解壓實作；換了條件之後，最快的做法在測過的範圍內都沒換（§4、§5）。
2. **GPU 共用的取捨**：改用 MPS 讓送回和別的工作同時跑時，回答快了多少；若再限制別的工作、替送回保留更多 GPU，別的工作又慢了多少；以及登記時的判定（§6）。
3. **固定做法之上的增量**：在三份真實 trace 上，用完美預知（oracle）算出預測最多能省多少，並在 GPU 上檢查模擬；其他機制各自和事先登記的門檻比（§7、§8）。
4. **決策流程**：把三個答案依序整理成設計者可以照做的流程（§9）。

2 背景：KV 從哪裡回來、要等多久

本節說明要選的東西：KV 有多大、停在哪、回來時有哪些送法（圖 2）。

KV cache 是什麼。模型讀過的每個 token 都會留下一組中間結果，叫 KV；下一則訊息要接著這些結果算，才不用重讀整段對話。KV 隨對話變長：32K token 時，Llama-3.1-8B 的 KV 有 4.0 GiB，Qwen2.5-7B 有 1.75 GiB（BF16）[M1]。

它為什麼離開 GPU。使用者在打字或在看回答時，這段 KV 只是佔著 GPU 記憶體。主流的服務引擎都會把它移到 CPU 記憶體；CPU 記憶體滿了，再移到固態硬碟（solid-state drive，SSD）或遠端儲存，或直接丟掉（§11）。本文的 SSD 級與 NFS 級（模擬網路檔案系統，Network File System）是限速模擬；這兩組頻寬常數沿用先前在另一個平台（AMD MI300X GPU）上量到的值，本文用的 NVIDIA H200 GPU 上沒有重量 [§0]。

使用者回來時。系統有三種送回方式。全部載入：把存的 KV 整段搬回。全部重算：在 GPU 上把整段對話重讀一次。Cake：GPU 從第一塊往後重算，同時從儲存層從最後一塊往前載入，兩邊相遇就完成（圖 2(a) 右上）。儲存層越慢，會合點越靠後，GPU 算的部分越多。

TTFT。從使用者送出下一則訊息，到看到回答第一個字的時間。它包含送回 KV 的時間，加上算新訊息的時間（預填，prefill）；在忙的服務裡還有排隊。§4 的地圖只量送回這一段。

還不知道的事。送回方式、格式、GPU 分配、預測與其他機制可以疊在一起（圖 2(b)），但哪個組合在哪種條件下最好、再加東西值不值得，沒有一張共同的表（§11）。

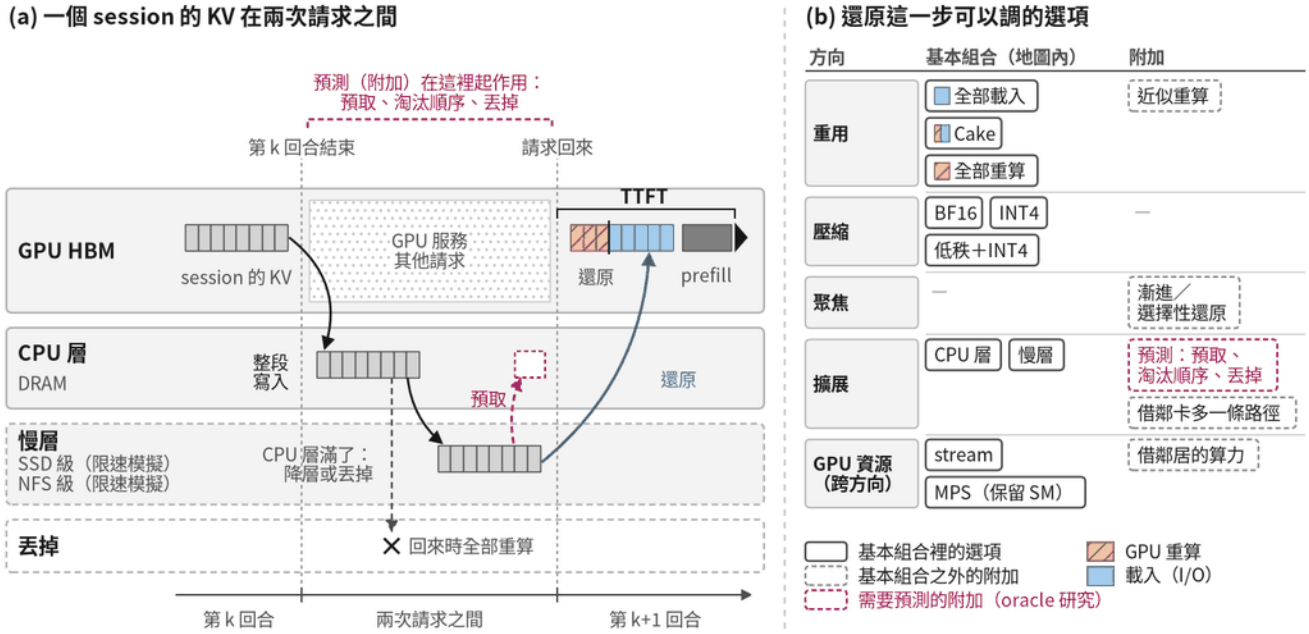


圖 2　KV 在兩次請求之間停在不同的層，回來時再送回 GPU；送回的時間算在使用者等的 TTFT 裡。

怎麼讀：(a) 由左到右是時間，由上到下是存放的層，最上面是 GPU 的高頻寬記憶體（high-bandwidth memory，HBM）；一個 session 的 KV 在回合結束時寫進 CPU 層，CPU 層滿了再降到慢層或丟掉，請求回來時送回 GPU。(b) 列出送回這一步可以調的選項：實線框是 §4–§6 比較的基本組合，虛線框是 §7–§8 量的附加，紅色虛線框是需要預測的附加。**代表什麼**：使用者等的時間裡有一段是送回 KV；本文後面的每個選擇，都是在縮短這一段。

本節的結論：要選的是送回方式、格式與 GPU 分配；下一節說明我們怎麼量、怎麼判斷值不值得。

3 方法：怎麼判斷「值得」

送回的組合在 GPU 上直接量；GPU 共用的端到端與預測用 trace 模擬，再在 GPU 上檢查模擬（圖 3）。

3.1 兩種量法與「值得做」的標準

- **GPU 微基準**〔實測〕：一次送回一段 32K 的 KV，量送回時間（不含新訊息的 prefill 與排隊）。每種做法重複 10 次（CPU 層、b32、NFS 級）或 30 次（b16、b48），取中位數 [M1]。b16／b32／b48 是把頻寬限到讓 Cake 在 64 塊的第 16／32／48 塊會合的慢層，Llama 為 8.83／2.49／0.722 GiB/s [§0]。§4、§5、§6 的微基準與 §8 的機制用這種量法。
- **trace 模擬**〔模擬〕：照真實 trace 的時間一次送一個請求，算回來請求的平均 TTFT；成本常數來自 GPU 微基準 [§0]。模擬是確定性的，每格跑一次。§6 的端到端與 §7 的預測用這種量法。

「值得做」的標準在看到任何數字之前就寫進登記檔：一個做法要快超過 5% 才算有改進（該實驗的量測雜訊比 5% 大時，就用雜訊當門檻；這個時間門檻下稱 δ^*），兩種做法差不到 5% 算平手；換格式時，答對率比 BF16 低不能超過 2 個百分點（看 95% 信賴區間〔confidence interval，CI〕的下界；這個品質門檻下稱 ε）；預測最多能省的比例要 ≥15% 才算值得做、全部 <5% 算死路，中間算可能 [§0]。5% 與 15% 沿用研究計畫一開始就定好的停損規則 [§0]。

我們用自己的 harness（量載入、重算與 Cake 的程式），而不是開源的服務引擎 vLLM、LMCache 或 SGLang，因為要能分開控制載入、重算與會合點。harness 的重算比 vLLM 慢（用 PyTorch 的 scaled dot-product attention〔sdpa〕時 1.75／1.76 倍，用 FlashAttention-3〔fa3〕時 1.30／1.32 倍）；模擬裡把重算加速到 vLLM 級，預測的判定不變 [E1、E2、B7]。

表 1　設定（只用 H200）

項目	值	證據
平台	一張 NVIDIA H200（143,771 MiB，PCI Express〔PCIe〕Gen5 x16），不做張量平行（tensor parallelism，TP）；vLLM 0.28.0、torch 2.13.0+cu130	§0
模型	Llama-3.1-8B-Instruct、Qwen2.5-7B-Instruct-1M	§0
格式	BF16；INT4（KIVI 式的量化）；低秩＋INT4（xKV 的做法，只有全部載入的路徑）	M1、M2
GPU 忙	地圖：合成的背景負載（GPU 一直滿載，duty 1.0），不是 MPS；§5 與 §6：像真實服務的背景（分塊預填〔chunked prefill〕、預填與逐字生成〔decode〕混合）	M1、M3、M4
CPU 層	地圖：真實的主機到 GPU 傳輸（host-to-device，H2D，用鎖頁記憶體）；模擬：讀 43.5888、寫 41.1687 GiB/s	M1、§0
限速慢層	b16／b32／b48：Llama 8.83／2.49／0.722 GiB/s，Qwen 4.33／1.23／0.358 GiB/s；NFS 級 0.331 GiB/s；SSD 級 6.981 GiB/s 只用在模擬	§0、M1
trace	Mooncake（FAST 2025 公開的使用紀錄）conversation 12,031 請求、toolagent 23,608 請求（各從一小時的線上請求抽樣）；FreeInference（公開的 agent 服務使用紀錄）前 6 週 129,717 請求（2026-05-17～06-27）；另一週 2026-07-26～08-01 只用於「知道誰不回來」	§0、B3-X5
品質	主測驗：32K、4 種任務共 192 題、seed 67000；多鍵大海撈針（needle in a haystack，NIAH）與 LongBench 長文測驗另報	D2-Q、M2

Mooncake 的 trace 引用 FAST'25；FreeInference 以 CC-BY-4.0 釋出，照資料卡引用 Nixon et al.（*From Requests to Sessions*，2026）。FreeInference 的 block 是 16 token：我們把連續 32 塊併成一塊 512 token，並用 token 數驗算 block 數。

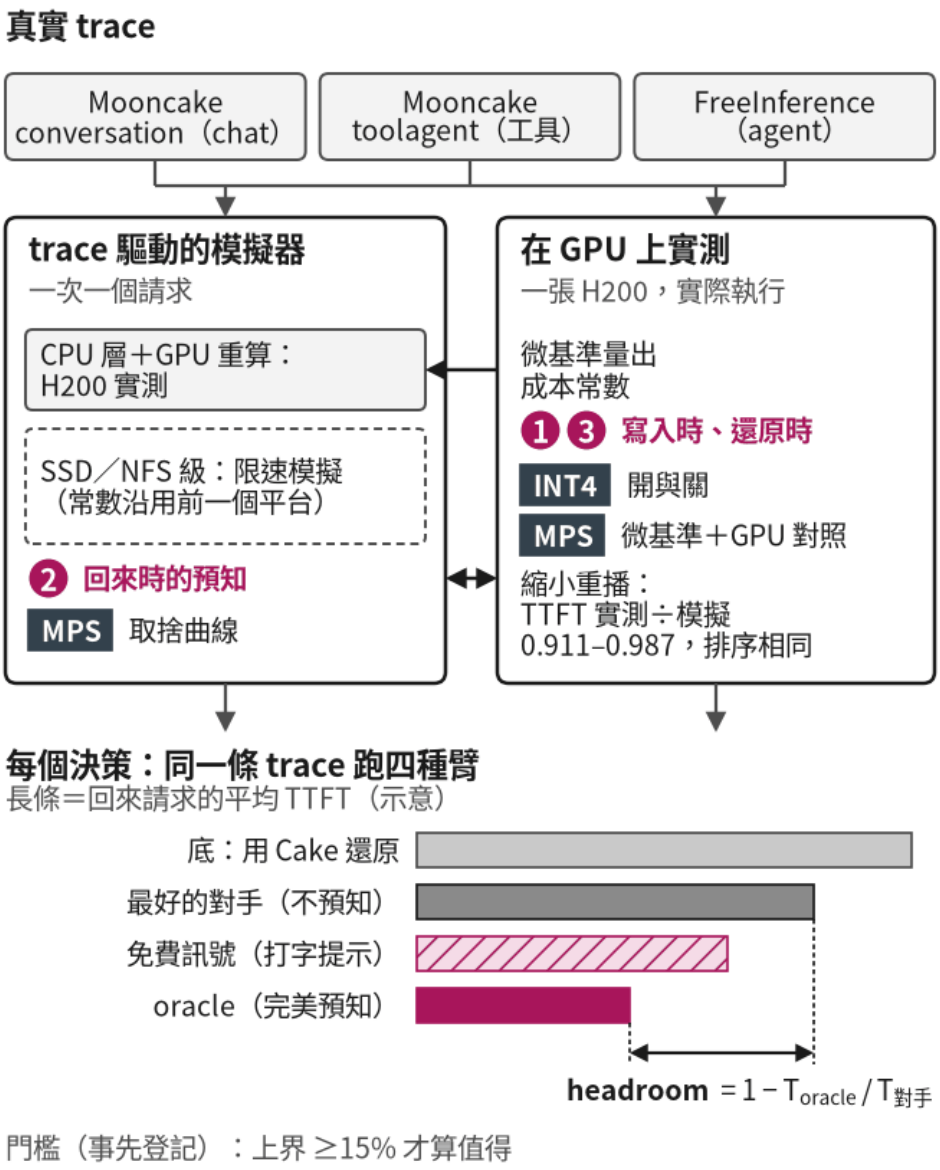


圖 3　方法：送回的組合在 GPU 上直接量；預測用 trace 模擬，拿完美預知和不預知的最好做法比，再用縮小的 GPU 重播檢查模擬。

怎麼讀：上方是三份 trace，左邊進模擬器、右邊在 GPU 上實際跑；中間的長條是示意，依序是不預測的底、最好的對手、免費訊號與完美預知（oracle），oracle 比最好的對手少花的比例就是預測最多能省多少。**代表什麼**：預測的價值是用「完美預知能省多少」來量；如果連完美預知都省不多，就不必蓋預測器。

3.2 預測的底、oracle 與公式

底：預測模擬的起點是一套固定、不預測的做法，接近 §4 的表在「逐塊解壓」時的答案 [§0]。慢層用 INT4＋Cake；CPU 層用 Cake，而逐塊解壓時 INT4 在 CPU 層不比 BF16 快 [D2]，所以 CPU 層存 BF16。和 §4 不同的有兩點：沒有 MPS；Mooncake 的 36 格主結果裡，19 格在地圖外的 SSD 級，另外 17 格在 b32 [B9]。寫入時的預知另在 GPU 上量，對照的做法是「延後到 CPU 層滿了才決定」[A1'']。

上界的意思：oracle 是同一個策略，只多了某一個未來量的完美資訊，所以它界住的是「把這個量的預測器放進同一個策略」最多能省多少，不是所有策略的極限。它也不是嚴格的上界：NFS 級有 4 格打字提示預取比 oracle 預取還快 [C2]。

一塊的載入時間：位元組 b 除以那一層的頻寬 B，INT4 再加解壓時間 d；那一層正在寫入時，讀取再乘量到的 k_{read} [§0]。

$$\ell_{\text{BF16}} = \frac{b_{\text{BF16}}}{B_{\text{CPU}}}, \qquad \ell_{\text{INT4}} = \frac{b_{\text{INT4}}}{B_{\text{層}}} + d \tag{1}$$

Cake：有存的前綴是第 0 到第 $L-1$ 塊；每一步讓落後的一邊前進一塊，兩邊在第 m 塊相遇，送回時間取兩邊較晚的那一個。$f(i)$ 是在 GPU 上重算第 i 塊的時間；H200 上量到第 79 塊（約 40K token），之後線性外插 [§0]。

$$T_{\text{restore}} = \max\left(\sum_{i=0}^{m-1} f(i),\ \sum_{i=m}^{L-1} \ell(i)\right) \tag{2}$$

模擬的 TTFT 是送回時間，加上重算沒存的塊與新訊息；策略 π 的成績是所有回來請求 R 的平均。

$$\text{TTFT}_r = T_{\text{restore}} + \sum_{i=L}^{n_r-1} f(i), \qquad T(\pi) = \frac{1}{|\mathrm{R}|}\sum_{r\in\mathrm{R}} \text{TTFT}_r(\pi) \tag{3}$$

四種回來時的 oracle 從 trace 讀「下一個讀到這個 session 的請求」，每個只改一個決定，其他照最近最少使用（least recently used，LRU）的規則 [§0]：

- **預取**：慢層讀取線空著時，先把最早回來的 session 的後段以 INT4 搬進 CPU 層。
- **淘汰順序**：CPU 層滿時，先降級最晚回來的 session（Belady 演算法：淘汰最晚才會再用到的資料；或把距離乘上能空出的位元組），每格取較快的一個。
- **回合結束就丟、輪到淘汰時才丟**：不再回來的 session 在這兩個時間點直接丟掉。截尾 H 時，離 trace 結尾不到 H 秒的決定一律當「會回來」。
- **知道誰不回來**（另列）：替每個 session 選去處（CPU BF16、CPU INT4、慢層或丟掉）；對手是最好的固定規則，任一主格 ≥15% 就算值得做。

上界是 oracle 比最好的同動作對手 S_c 少花的比例；同一組設定要有兩種以上的 CPU 容量 ≥15%，才算值得做。Mooncake 沒有 session 欄位，我們用兩種規則重建，每格取較小的上界 [§0]。

$$h_c = \frac{T_{\text{rival}} - T_{\text{oracle}}}{T_{\text{rival}}}, \qquad T_{\text{rival}} = \min_{\pi\in\mathrm{S}_c} T(\pi) \tag{4}$$

打字提示的提前量：這一輪新輸入的 token 數 Δ_r 扣掉 10 個模板 token，換成字元數（每 token c_{tok} 個字元），除以桌機打字速度 51.56 WPM（每分鐘字數，words per minute；Dhakal et al.），再以兩請求之間的空檔為上限（每個輸出 token 的時間〔time per output token，TPOT〕取 50 ms）[§0]。

$$L_r = \min\left(\frac{\max(\Delta_r - 10,\ 0)\cdot c_{\text{tok}}}{51.56 \times 5/60},\ \text{think}_r\right) \tag{5}$$

慢層的寫入負載 ρ_w＝寫入的忙碌時間 ÷ trace 長度；$\rho_w \geq 1$ 的格就是寫入飽和 [§0]。

3.3 模擬的排序與正負號在 GPU 上站得住

我們把 Mooncake conversation 依 session 抽 1/8，在 H200 上用真的搬運與重算重播 12 個乾淨的 run（SSD 級、Llama、CPU 16／32 GiB）（圖 4）[B6]：

- 實測÷模擬＝0.911–0.987，10 格全在 ±10% 內 [B6]。
- 模擬差 ≥5% 的 15 對策略，排序全同；8 個上界的正負號全同 [B6]。
- 重播一次只送一個請求，TTFT 不含排隊；含排隊時上界變小（§10）。

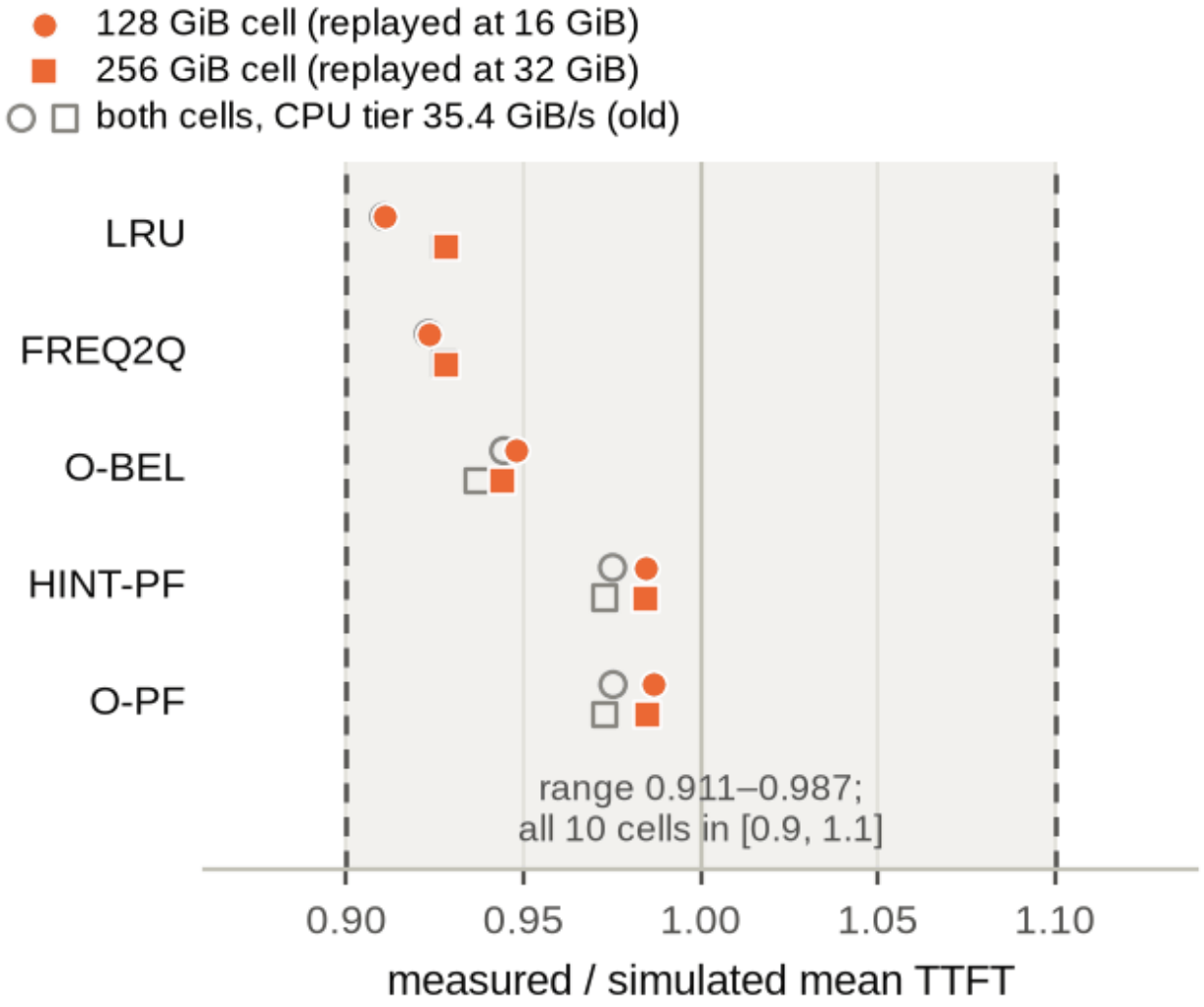


圖 4　模擬比實測略慢，但都在 ±10% 內，排序與正負號一致。〔實測〕÷〔模擬〕，TTFT 不含排隊；灰色空心是舊 CPU 常數的重播 [B6]。

怎麼讀：每一列是一種策略（O-BEL＝完美淘汰順序、O-PF＝完美預取、HINT-PF＝打字提示預取、FREQ2Q＝依使用次數淘汰的對手），橫軸是 GPU 實測 ÷ 模擬的平均 TTFT，1.0 表示完全一樣，灰底是 0.9–1.1。**看到什麼**：10 個實心點落在 0.911–0.987 之間，全部在灰底內；模擬給的策略排名和實測相同 [B6]。**為什麼**：模擬為什麼略慢，我們沒有查證。**代表什麼**：用模擬比較策略、判斷值不值得是可信的；但這只驗了 conversation、SSD 級與 Llama，而且不含排隊。

3.4 偏離與量測防護

和登記不同的做法寫成「偏離」；會碰到報告數字的列在下表，完整清單在匿名 artifact（審查期間不附連結）。沒過門檻的檢驗也照列（§5、§7、§8）。這是共用叢集：每個量時間的 run 都有 GPU 監看，同一張卡出現外來 process 就作廢重量 [R-T1G、E4]。

偏離	影響	出處
FreeInference 在預測實驗登記之後、看結果之前才加入；原登記要求總結以只用 Mooncake 的版本為準	FreeInference 的結論屬於偏離；H200 上的重跑另有登記	X1 登記 §2.9、T5
截尾 *H* 是登記之後加的穩健性檢查；登記規定以較保守的版本為準，截尾版較保守，所以當 Mooncake 的主結果	比原登記保守	X1 登記 §4.7、B9
換成 H200 的 CPU 層常數後，排隊模擬的一項自我驗證不適用，不再當停止條件	排隊的數字照登記算	T1 偏離 1

偏離	影響	出處
MPS 的 GPU 檢查因時限拆成兩個 job；15 個 run 都在同一張卡	無	E2E GPU 偏離 1
同節點別人 job 的開始與結束：照登記用全部量測單位，另報拿掉後的結果	判定不變	T1G 附加 1

3.5 AI 使用揭露

harness、模擬器、在叢集上執行實驗與分析腳本，由 AI coding agent（Claude Code，模型 Claude Opus 5.5）撰寫並執行。研究問題、事先登記的門檻、每個 go／no-go 決定與解讀由作者負責。每個數字都對應一條指令與一個輸出檔；判準先登記；模擬經 GPU 重播檢查（§3.3）。〔待決：作者人工檢查核心程式那一句〕

本節的結論：判準與量法都先定好；下一節用 GPU 微基準回答第一個問題的前半：哪個組合最好。

4 哪個還原組合最好：慢層用 INT4＋Cake，CPU 層看解壓實作〔實測〕

不加預測時，最快的組合隨層、GPU 忙閒、模型與 INT4 的解壓實作變；在 b32、b48、NFS 級每一格都是 INT4＋Cake [M1]（表 2、圖 1）。

圖 1 怎麼讀（圖在一頁看懂之後）：每一列是一種「層 × GPU 忙閒」，每一欄是一種送回方式與格式；格子裡的數字是送回時間除以該列最快的時間，粗框是最快。**看到什麼**：在慢層，BF16 整段載入最慢，例如 Llama、NFS 級、GPU 閒時要 12.082 s，INT4＋Cake 只要 1.2412 s [M1]；在 CPU 層、GPU 忙時，最快的是 INT4 整段載入 [M1]。**為什麼**：INT4 每塊的位元組小 3.2 倍 [D1]，所以載入那一邊更快；Cake 的送回時間是「前段重算」與「後段載入」兩者中較晚的那一個（§3.2 公式 (2)），載入變快，會合點就往前移，兩邊一起縮短。CPU 層本身搬得很快，所以解壓的做法決定快慢：同樣是 Llama、CPU 層、GPU 忙的 INT4 整段載入，一次拆完要 0.0342 s，逐塊解壓要 0.2325 s [M3-g]。整段重算最慢，因為 32K 的 64 塊全部重算要 1.85–4.05 s [M1]。**代表什麼**：先看 KV 停在哪一層，再看解壓實作，就能查出該怎麼送回。

表 2　32K 單次還原的最快組合（格式要在主測驗過 ε；領先＝比第二好的做法快多少）

層（INT4 的解壓）	GPU 閒	GPU 忙	證據
CPU 層（一次拆完）	Llama：低秩＋INT4**；不算低秩時 INT4 全部載入與 Cake 平手。Qwen：INT4 全部載入，領先 37.9%	INT4 全部載入；Llama 領先 23.1%（這格的雜訊大，28.4%），Qwen 54.2%	M1
b16（逐塊解壓）	Llama 平手；Qwen INT4 全部載入，領先 7.1%	INT4＋Cake，領先 9.4–10.4%	M1
b32（一次拆完）	INT4＋Cake，領先 25.4–25.6%；Llama 的低秩**更快	INT4＋Cake，領先 11.0–12.0%	M1
b48（逐塊解壓）	INT4＋Cake，領先 37.4–37.9%	INT4＋Cake，領先 36.2–36.3%	M1
NFS 級（一次拆完）	INT4＋Cake，領先 27.2–40.7%；Llama 的低秩**更快	INT4＋Cake，領先 37.0–37.4%；Llama 的低秩**更快	M1
SSD 級	沒量	沒量	M1

** 只有 Llama 的每題奇異值分解（singular value decomposition，SVD）配置在主測驗過 ε；寫入時每個 32K 請求要多花 3.24–5.31 s 做 SVD，不算在還原時間裡 [M1、M2]。領先只在 BF16／INT4 的做法之間算。Qwen 的 INT4 在多鍵 NIAH 的 Cake＋GPU 忙讀法沒過 ε（−0.8 pp，CI 下界 −2.2），圖 1 用 † 標出 [M2]〔待決〕。

- **CPU 層：解壓實作比送回方式更重要**。同一個組合（CPU 層、GPU 忙、INT4 全部載入），一次拆完和逐塊解壓差 6.79–6.85 倍 [M3-g]。一次拆完時，GPU 忙的 INT4 全部載入比 Cake 快 23.1%（Llama）與 54.2%（Qwen）[M1]；逐塊解壓時，全部載入反而比 Cake 慢 1.52–3.52 倍 [E3]。所以在 CPU 層查表時，要先知道自己用的是哪一種解壓實作。
- **保守的預設**：不確定實作時，一律用 Cake。在成本模型的 24 個測試格裡，一律 Cake 平均只比最好的多 6.7%、最差 60.5% [M3-f]；但在一次拆完的實作下，CPU 層 GPU 忙時 Cake 是最快做法的 1.30 倍（Llama）與 2.18 倍（Qwen）（圖 1）。
- **全部重算**在 32K 的每一格都不是最好：GPU 閒時要 1.85–2.11 s，忙時 3.56–4.05 s [M1]。
- **只准 BF16 時**，CPU 層 GPU 忙要改用 Cake（Llama 0.121 vs 0.238 s）；所有慢層也都用 Cake [M1]。

- **低秩＋INT4 的代價**：GPU 忙時，低秩＋INT4 在 CPU 層與 b32 至少要約 0.71–0.77 s，比 INT4 慢；Qwen 沒有在主測驗過 ε 的配置 [M1、M2]。

本節的答案：只要知道 KV 停在哪一層、GPU 忙不忙、用哪個模型、INT4 用哪種解壓實作，就查得到該怎麼送回；慢層用 INT4＋Cake，CPU 層照實作選整段載入或 Cake，b16 上兩者差不到 10.4% [M1]。下一節檢查換了條件之後，這張表會不會變。

5 這張表穩不穩：在測過的條件下不翻轉〔實測〕

§4 的表是在固定條件下量的；本節換掉重算速度與背景負載，最快的做法在測過的層上都沒換 [A5、M3]（表 3）。

表 3　表 2 的穩定性

檢驗	涵蓋	結果	證據
重算換成更快的 fa3	CPU 層、b32、NFS 級（24 格＝2 模型 × 3 層 × 閒忙 × 2 格式）	0 格翻轉，1 格從平手變成全部載入；地圖對應的 12 格最快做法都不變；b16、b48 沒測	A5、M1、M3
合成的忙換成像真實服務的背景	2 模型 × CPU 層與 b32（這裡的 b32 頻寬和地圖不同）；沒有 b16、b48、NFS、SSD 級	可判定的線 0 條翻轉；2 條因雜訊太大沒判；報告標為暫定	M3-b
線上選擇器（CPU 層、GPU 忙；逐塊解壓）	12 格 × 2 個 seed	30–100% 的次數選了全部載入，在這個實作下比 Cake 慢 1.52–3.52 倍；換 seed 部分重現	M3、E3
用微基準常數的成本模型（逐塊解壓）	24 個測試格	平均多花 33.4%，最差 256%	M3-f

後兩列的錯，有一部分來自實作：它們的「全部載入」是逐塊解壓；換成表 2 的一次拆完，CPU 層最快的反而是全部載入 [M3-g]。

本節的答案〔判讀〕：在測過的條件下，表 2 不隨重算速度或背景負載改變，所以系統只要照一張固定的表選，表裡分開兩種解壓實作；在每個請求上現場量測或用成本模型挑，並沒有比較好 [A5、M3]。下一節問：GPU 同時在跑別的工作時，又該怎麼分。

6 GPU 共用時怎麼分：改用 MPS 回答快很多，保留越多別的工作越慢〔模擬＋GPU 檢查〕

§4 假設送回時 GPU 只做這件事；本節讓 GPU 同時跑別的工作。改用 MPS 讓兩者同時跑、先不限制別的工作時，回答比單純共用快 30.7–62.9%，別的工作的處理量只少 0–6.5 個百分點；若再限制別的工作、替送回保留更多 GPU，回答更快，但別的工作明顯變慢 [S2、M4]（表 4、圖 5）。

MPS 與 stream 共享是什麼：NVIDIA 的多行程服務（Multi-Process Service，MPS）讓多個程式同時在同一張 GPU 上執行，並能限制每個程式最多使用多少計算單元（streaming multiprocessor，SM，GPU 上的計算單元）。我們讓送回的程式不受限，只限制別的工作：不設上限，或最多用 87%、75%、50% 的 SM；限得越多，等於替送回保留越多 SM。對照組是 stream 共享：兩邊的工作各放在一條 CUDA stream（GPU 上的工作佇列）、不設上限，由 GPU 交錯執行 [M4]。

這裡是模擬的設定：慢層用 §4 的 INT4，對照組是 stream 共享，底另含打字提示預取（§7.4）[M4]。我們用 B 表示別的工作的處理量，是它獨自用 GPU 時的幾成；就算只是單純共用 stream，B 也只有 0.697–0.957，6 組裡只有 1 組達到登記要求的 0.95 [M4、S2]。

表 4　MPS 的取捨（gain＝回答的等待比單純共用縮短多少；背景 87%＝別的工作只能用 87% 的 SM）〔模擬〕

模型, 背景, trace	stream：B	MPS 不設上限：gain／B	背景 87%	背景 75%	背景 50%
Llama, chunked prefill, conversation	0.801	50.4%／0.777	71.7%／0.668	77.8%／0.643	沒量
Llama, chunked prefill, FreeInference	0.937	62.9%／0.937	76.0%／0.772	79.5%／0.752	沒量
Llama, mixed, conversation	0.697	33.2%／0.632	33.7%／0.607	36.3%／0.602	44.7%／0.470
Llama, mixed, FreeInference	0.929	39.8%／0.889	39.9%／0.829	42.0%／0.811	46.8%／0.622
Qwen, mixed, conversation	0.701	30.7%／0.648	32.2%／0.624	33.3%／0.607	42.4%／0.487
Qwen, mixed, FreeInference	0.957	31.5%／0.915	32.2%／0.866	33.4%／0.803	41.0%／0.630

Qwen 加 chunked prefill 背景沒有量 [M4]。

- **登記的端到端判定**：登記時要求兩件事同時成立：回答快 15% 以上，而且 B ≥0.95。可判的 24 格沒有一格做到，判定不成立 [S2]。
- **最快的 MPS 等級是不是固定的**：登記的判定是「不成立」，原因只有 VQ 解碼這一個動作的最佳等級會變；其他 20/22 個（模型, 動作）在微基準裡，最快的等級都一樣 [M3-c]。
- **微基準**〔實測〕：有 18 種情況下，用 MPS 送回比單純共用快 30% 以上，別的工作仍保有 0.91 以上的處理量 [S1]。
- **GPU 檢查**：登記判定不成立（MPS-GPU-INVALID）。可比的 7 格方向都和模擬一樣，但 GPU 上回答變快的幅度比模擬多 4.6–31.3 個百分點；所以「保留越多、回答越快、別的工作越慢」這個趨勢可信，確切的數字要在自己的 GPU 上量 [S3]。

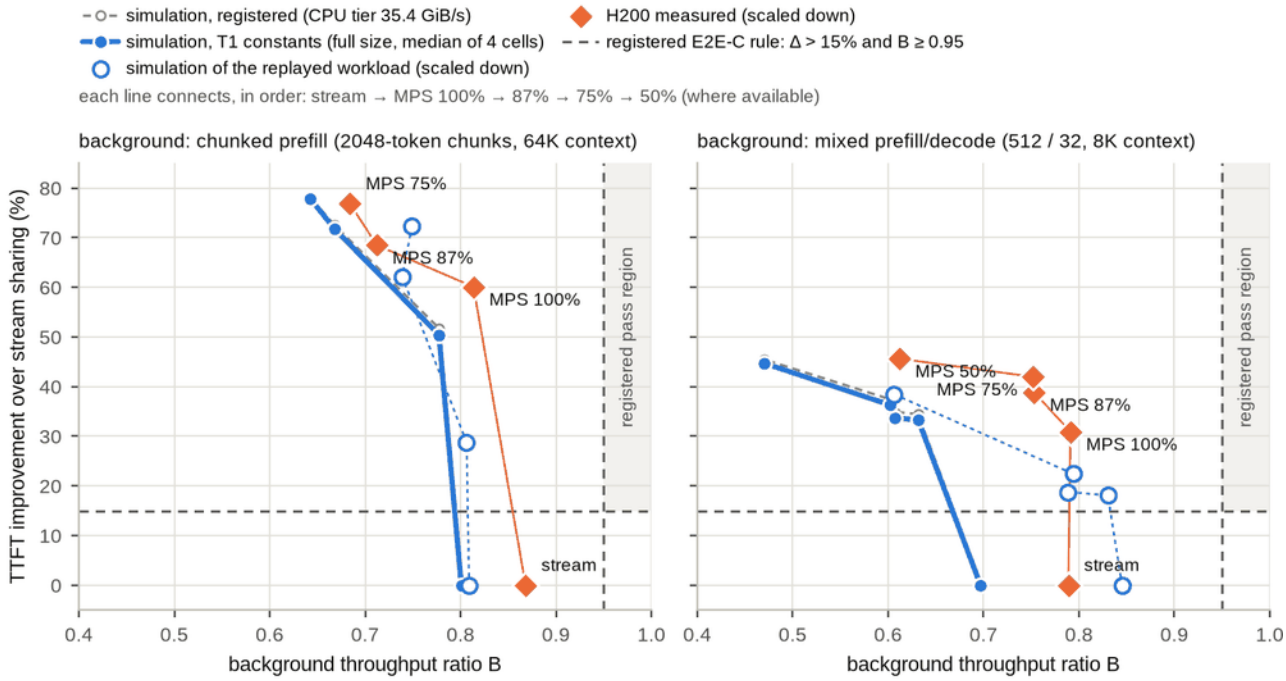


圖 5　替送回保留越多 GPU，回答越快、別的工作越慢；沒有一點同時達到登記的兩個要求。〔模擬〕＋〔實測〕左：Llama、chunked prefill、conversation；右：Llama、mixed、conversation。藍實線＝H200 常數的模擬，灰虛線＝登記版常數，空心圈＝縮小工作的模擬，◆＝H200 實測 [S2、S3、M4]〔待決〕。

怎麼讀：橫軸是別的工作保有的處理量 B，縱軸是回答的等待比單純共用縮短多少；每條線由右下往左上依序是單純共用、MPS 不設上限、背景 87%、75%、50%，右上角的灰區是登記要求的區域。**看到什麼**：以左圖為例，只是改用 MPS 時回答快 50.4%、B 從 0.801 降到 0.777；限到背景 75% 時回答快 77.8%，B 降到 0.643 [M4]。**為什麼**〔判讀〕：MPS 讓送回拿到固定的一份 SM，不必和別的工作的 kernel 輪流；保留得越多，別的工作能用的就越少。**代表什麼**：先決定別的工作最少要保有多少處理量，再挑 MPS 等級。

本節的答案〔判讀〕：先決定別的工作最少要保有多少處理量，再在自己的 GPU 上試兩三個 MPS 等級。在模擬裡，只是改用 MPS（不限制別的工作）時，別的工作的處理量只少 0–6.5 個百分點；要再快，就得限制別的工作，讓它慢下來 [S2、S3、M4]。下一節回到 §5 的發現：既然一張固定的表就夠，預測還能再省多少。

7 在固定規則之上，預測還值多少：要看 trace〔模擬；寫入時是實測〕

§5 顯示一張固定的表就夠，所以本節問預測能不能再省：寫入時就決定放哪層，在 GPU 上量不到收益；回來時的預測在 FreeInference 前 6 週上連完美預知都快不到 15%，在 Mooncake 上預取與淘汰順序仍有空間 [A2、B3-T5、B9]。 預測模擬的底見 §3.2。

7.1 寫入時的預知：量不到收益〔實測〕

依塊在對話裡的位置，在寫入時決定放哪層，對 TTFT 沒有可量到的收益；依位置決定精度，也沒有穩定的品質差距 [A1''、A2、A4]（表 5）。

表 5　寫入時預知的 GPU 實測（對照＝延後到 CPU 層滿了才決定）

檢驗	結果	判定	證據
依位置決定放哪層，重算用 fa3／sdpa	TTFT 差 −0.37～+0.01%／−0.02～+1.16%，門檻 5%，每點 6 次重複	量不到	A1''、A1'
寫入時決定，同時贏兩個對手各 ≥5%	16 個 GPU 列 0 列過；模擬裡存活的設定上 GPU 確認 0/2	量不到	A2、A3
依位置降精度（品質檢定）	前、中、後段的品質差方向不一，沒有穩定的差距	量不到	A4、A4'

〔判讀〕Cake 從前段重算，還原時大多不讀前段，所以前段放哪層、存什麼精度影響不大 [A4]。寫入准入對 SSD 寫入量與壽命的好處不在本文的量測範圍。

7.2 FreeInference：命中率已經很高，回來時的上界就低

前 6 週、CPU ≥128 GiB 時，四種回來時上界都 ≤0.30%；在測過的 16–64 GiB 也沒有一格 ≥15% [B3-X1、B3-T5]（表 6、圖 6）。〔待決〕

表 6　FreeInference 的上界（前 6 週，最後一列除外；每格取慢層 × GPU 間／忙的最大值）

上界	CPU 16–64 GiB	CPU 128–1,024 GiB	證據
預取	≤4.89%	0.22%	B3-T5；圖 7 sidecar
淘汰順序	≤11.81%（≥5% 只在 NFS 級、16–32 GiB）	0.30%	B3-T5、B3-X1
丟棄（兩種）	≤0.65%	0.17%	B3-T5；圖 7 sidecar
判定（四種上界）	可能（4 格 ≥5%、0 格 ≥15%）	死路	B3-T5、B3-X1
知道誰不回來（另一週 2026-07-26～08-01）	沒量	2.44%（死路）	B3-X5

- **為什麼低**：不是因為 CPU 對工作集比較寬裕：CPU 128 GiB 只放得下 FreeInference 工作集的 0.78–8.06%，Mooncake 則是 1.20–1.28% [§0]。FreeInference 的回來請求在 128 GiB 時有 99.29% 命中 CPU 層，沒有差距可補 [B3-T4]；Mooncake 同容量的命中率本文沒有算 [§0]〔判讀〕。
- **小 CPU 時**：那 4 格全是淘汰順序＋NFS 級，最好的對手是 S3-FIFO（用三個先進先出佇列淘汰的策略）；對手換成 LRU，同樣的格看起來有 17.7–29.2%，多出來的來自 LRU 太弱 [B3-T5、B3-T4]。預取的 oracle 在小 CPU 時比對手慢（最低 −29.5%），所以這裡的上界可以是負的：它只預知回來時間，騰空間時照 LRU [B3-T5]〔判讀〕。
- **和 Liu 等人對照**（他們在同一條 trace 上比較淘汰策略，§11）：命中率差距與「沒命中就重算」的 TTFT 差距各自在小容量出現過，但沒有一個容量兩者同時越過登記的門檻 [B3-T4]。這不是他們的重現：block 粒度、併發與 oracle 都不同。

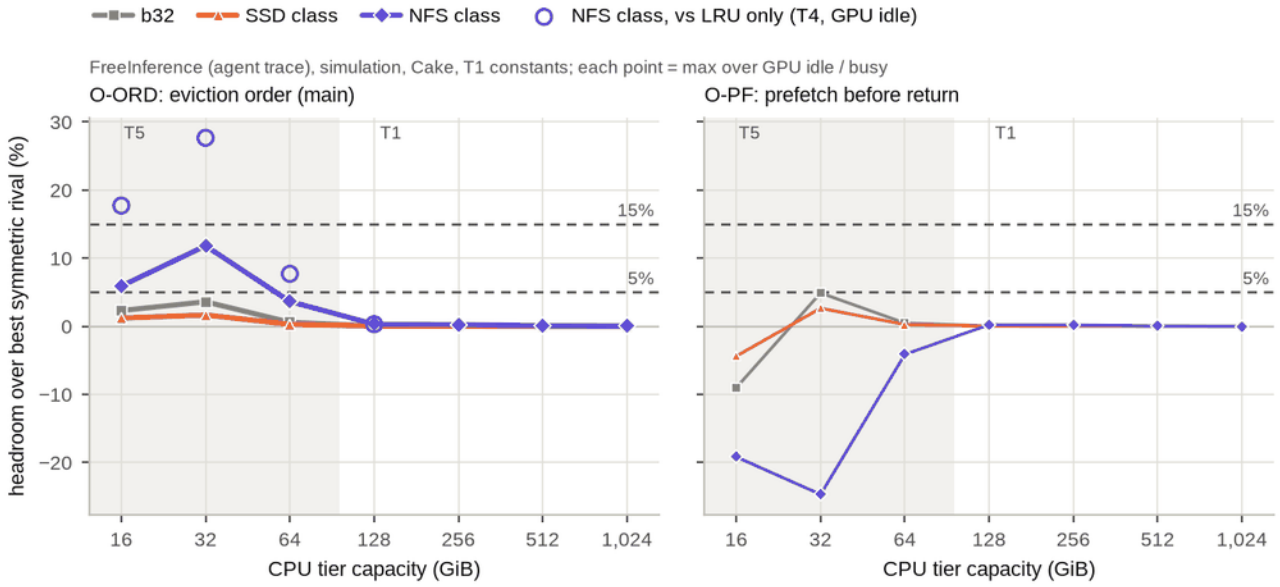


圖 6　FreeInference 前 6 週，淘汰順序與預取的上界在測過的 CPU 16–1,024 GiB 每一點都低於 15%。〔模擬〕實線對最好的對手；空心圈對 LRU，只在 16–64 GiB 高於實線；圖內 T5／T1 標示兩批模擬的容量範圍 [B3-T5、B3-T4]。

怎麼讀：橫軸是 CPU 層的容量，縱軸是完美預知比最好的不預知做法少花的比例；左圖是淘汰順序、右圖是預取，三條線是三種慢層，虛線是 5% 與 15%。**看到什麼**：每一點都在 15% 以下；最高的是淘汰順序在 NFS 級、CPU 32 GiB 的 11.81%，CPU ≥128 GiB 時都接近 0 [B3-T5、B3-X1]。**為什麼**：CPU 128 GiB 時，回來的資料已有 99.29% 在 CPU 層 [B3-T4]，完美預知也沒有東西可改；小容量時預取反而是負的（最低 −29.5%），因為它騰空間時照 LRU，把別的 session 擠到慢層 [B3-T5]〔判讀〕。**代表什麼**：在這類 trace 上，不必為「誰會回來」蓋預測器。

7.3 Mooncake：截尾後仍有 36 格 ≥15%

Mooncake 的兩條 trace 上，截尾並排除寫入飽和後，192 格裡仍有 36 格 ≥15%，CPU 到 512 GiB 都有；其中 17 格在 b32、19 格在 SSD 級、NFS 級 0 格，所以結論不只靠地圖外的層 [B9]（表 7、圖 7）。

表 7　Mooncake 的上界 ≥15% 的格數（每個容量 48 格＝2 條 trace × 4 種上界 × 3 種慢層 × GPU 閒／忙）

	CPU 128	256	512	1,024 GiB	合計
截尾 H ＝3,600 s，並排除寫入飽和（主結果）	16	12	8	0	36/192
不截尾（含丟棄格與寫入飽和格，只報告）	28	31	24	12	95/192

- **為什麼用截尾版**：trace 只有一小時，快到結尾的 session 看起來都不會再回來，「回合結束就丟」因此顯得很有用；不截尾的 95 格裡有 28 格是它，截尾後都是 0.0% [B2、B9]。
- **conversation**：代表格（SSD 級、GPU 閒）的預取上界 50.5%、淘汰順序 27.6／31.0% [B1]。預取的登記對手只有 LRU 與 LRU+RPF（預取最近用過的 session）；換成較快的 LRU2（看倒數第二次使用時間的 LRU）是 45.3% [B1-R]。
- **toolagent**：96 格裡 40 格 ≥15%，截尾並排除寫入飽和後仍 12 格 [B8]。
- **GPU 上仍在，但較小**：縮小重播上，預取對 LRU 是 17.9%，模擬是 24.2%；含排隊時只剩 8.1／4.7% [B6]。toolagent 沒有 GPU 重播。
- **知道誰不回來**：48 格裡 11 格 ≥15%，10 格在 CPU ≤256 GiB、全部在 b32／b48 [B5]〔待決〕。

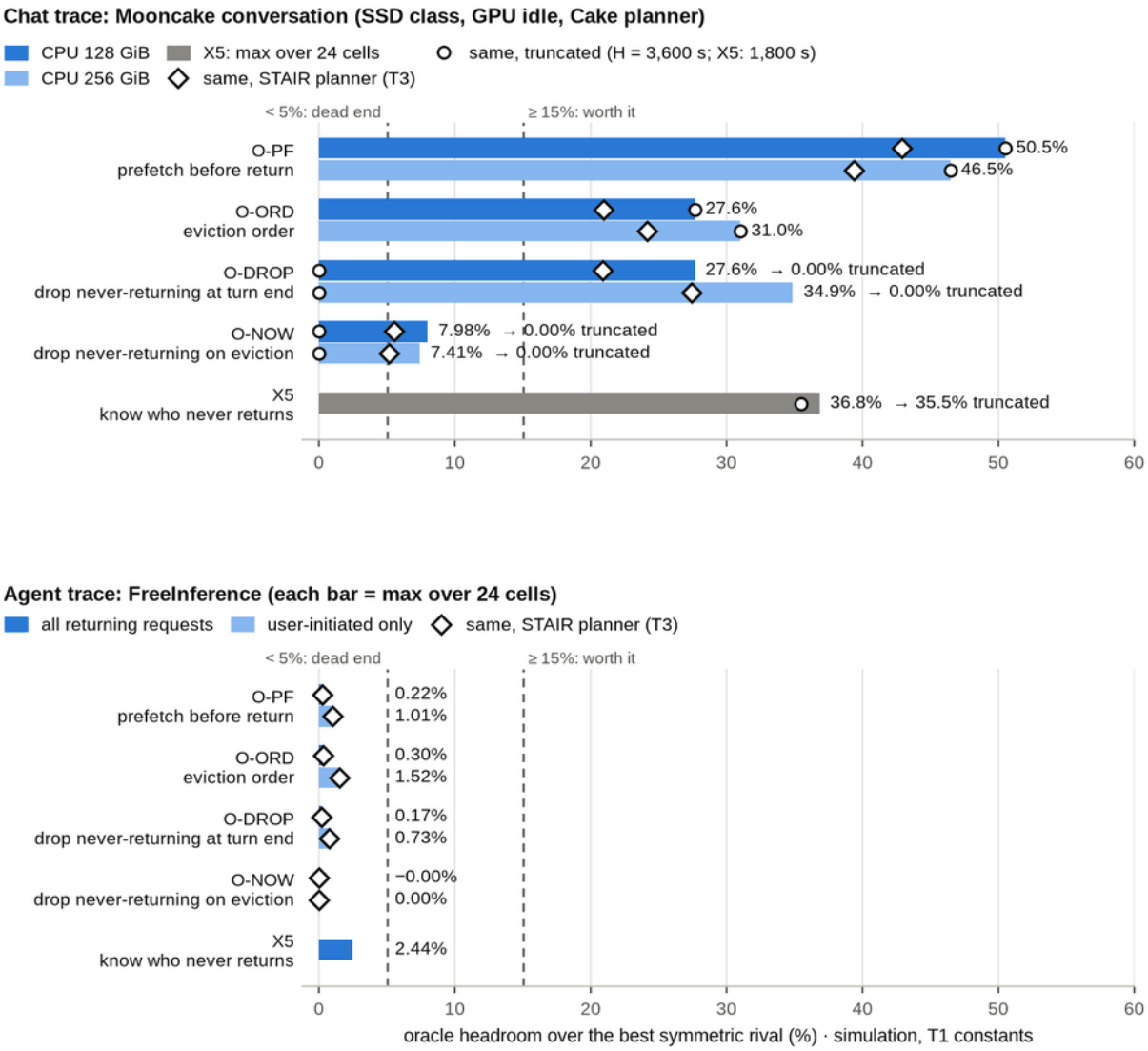


圖 7　同樣的完美預知，在 Mooncake conversation 的代表格上能省 15% 以上，在 FreeInference（前 6 週、CPU 128–1,024 GiB）上全部省不到 5%。〔模擬〕丟棄的上界大半來自 trace 太短，截尾後是 0.0% [B2]；X5 那根（2.44%）用的是另一週。圖內代號：O-PF＝預取、O-ORD＝淘汰順序、O-DROP／O-NOW＝回合結束就丟／輪到淘汰時才丟、X5＝知道誰不回來、◇＝更強的還原規劃器 [B1、B3-X1、B3-X5]。

怎麼讀：每根長條是一種完美預知比最好的不預知做法少花的比例；上圖是 Mooncake conversation 的代表格（SSD 級、GPU 閒），下圖是 FreeInference 的最大值，兩條虛線是 5% 與 15%。**看到什麼**：上圖的預取最高，CPU 128 GiB 時 50.5%，淘汰順序 27.6%；下圖每一根都在 2.44% 以下 [B1、B3-X1、B3-X5]。**為什麼**：FreeInference 的回來資料幾乎都已在 CPU 層 [B3-T4]；Mooncake 為什麼差這麼多，我們沒有同樣的命中率可比 [§0]。**代表什麼**：預測值不值得，要先看自己的 trace，不能從別人的 trace 推。

7.4 免費訊號：打字提示拿到預取的大部分

打字提示拿到預取上界的大部分；淘汰順序還沒有測過任何免費訊號 [C2-GPU、B1-ORD]（表 8、圖 8）。

提前量是合成的：Mooncake 沒有按鍵時間，我們用公式 (5) 算；這個提前量在 conversation、NFS 級上能藏住 62.2–72.0% 的慢層還原時間 [C1]。

表 8　打字提示預取拿到 oracle 預取改善的比例（Mooncake conversation）〔待決〕

設定	比例	證據
GPU 縮小重播，CPU 16／32 GiB〔實測〕	0.81／0.79	C2-GPU

設定	比例	證據
全尺寸模擬，SSD 級／b32／NFS 級的中位〔模擬〕	0.933／0.941／1.118	C2
全尺寸模擬，24 格的最小〔模擬〕	0.814	C2

- GPU 上，oracle 預取只比打字提示快 4.0／3.8%，不到 5% [C2-GPU]。比例 >1 的 4 格全在 NFS 級，多數寫入飽和，是較弱的證據 [C2、R-T3]。
- **淘汰順序沒有免費訊號**：打字提示只和預取比過；淘汰順序的上界沒有任何免費訊號比過 [B1-ORD]。

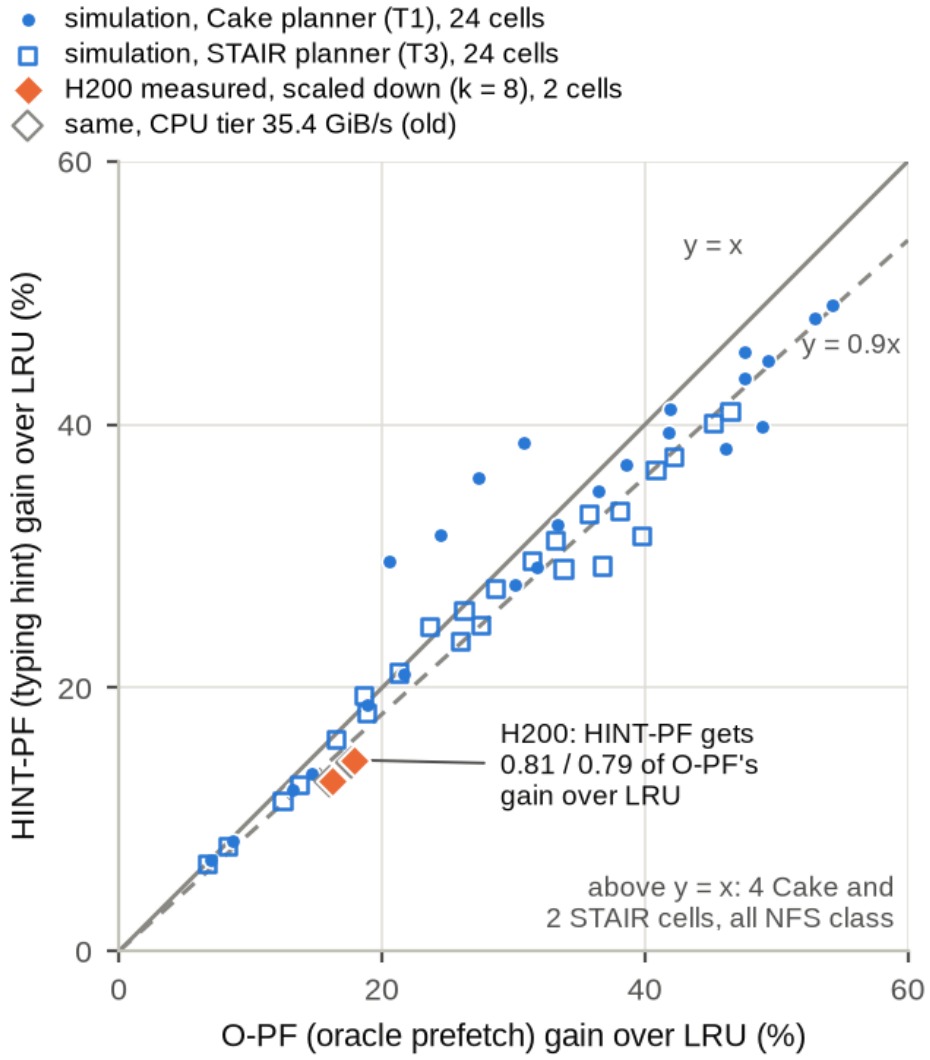


圖 8　打字提示拿到完美預取改善的大部分：模擬點貼著 y＝x，GPU 實測是 0.81／0.79。〔模擬〕＋〔實測〕；y＝x 以上的格全在 NFS 級；圖內 HINT-PF＝打字提示預取、O-PF＝完美預取 [C2、C2-GPU]。

怎麼讀：每個點是一種設定；橫軸是完美預取比 LRU 快多少，縱軸是打字提示預取比 LRU 快多少，點越靠近 y＝x，打字提示越接近完美。**看到什麼**：模擬點大多貼著 y＝x；GPU 縮小重播的兩點是 0.81 與 0.79 [C2、C2-GPU]。**為什麼**：使用者打字的時間夠長，能藏住 conversation、NFS 級上 62.2–72.0% 的慢層還原時間 [C1]，所以大多數資料在請求到之前已經搬好。**代表什麼**：要做預取，先用打字事件就夠，不必蓋預測模型；但這個訊號只幫得上預取。

本節的答案〔判讀〕：寫入時的預知不必做；回來時的預測要先在自己的 trace 上量上界，FreeInference 這類 trace 上不值得，Mooncake 這類 trace 上預取先用打字提示，淘汰順序還沒有免費訊號 [A2、B3-T5、B9、C2-GPU、B1-ORD]。下一節看預測以外的機制。

8 其他不靠預知的機制：只有多一條路徑過門檻〔實測〕

§7 量了預測；本節在 GPU 上把其他機制和各自登記的對照比，只有借閒置鄰卡多一條還原路徑過了門檻（表 9）。
INT4、低秩、MPS 與預測已在 §4、§6、§7 回答 [M5]。

表 9　其他機制的判定

方向	機制（做什麼）	判定	關鍵結果	證據
聚焦	漸進還原（先讀粗版、只精修重要的塊）	停止	比差不多位元組的均勻壓縮：Qwen 只好 1.28 pp（要 ≥2）、Llama 好 2.32 pp。登記的確認實驗以 Qwen 為準，Llama 只是輔助條件；兩個模型對 BF16 的 CI 下界（−3.87／−3.37 pp）也都低於 −2 pp	G-A0、M5-聚焦-a
重用	近似（稀疏）重算	停止	品質掉 8.64–14.56 pp（要 ≥ −2）	G-M12
壓縮	降級時才在 CPU 上壓縮	不成立	CPU 壓縮速度跟不上 KV 產生的速度，要在 GPU 上壓	M5-壓縮-d
擴展	借閒置鄰卡多一條還原路徑（鄰卡也從 CPU 讀，再經 GPU 之間的直連 NVLink 傳過來）	過	16 GiB 還原快 1.986 倍；只有 2 卡、鄰卡閒，鄰卡受的影響沒量	G-M06
GPU 資源（第五類）	借鄰居的算力（用 decode 用不完的算力做還原）	停止	45 個候選沒有一個過；別的工作的處理量只剩 0.35–0.74	G-M08

另有三個登記了但沒有完成判定：查表式 VQ 解碼、打字時挑塊、依注意力頭還原 [M5]。

本節的答案〔判讀〕：在固定的做法之上，單張 H200 上沒有其他機制能無條件地再省時間；多一條路徑過了門檻，但它需要一張閒著的鄰卡，值得在多卡上再量 [M5、G-M06]。下一節把三個答案依序整理成流程。

9 給設計者的決策流程

三個答案依序串成一個流程：先用 §4 的表選送回方式與格式，再照 §6 決定保留多少 GPU，最後才照 §7、§8 決定要不要加東西（圖 9）。〔判讀〕

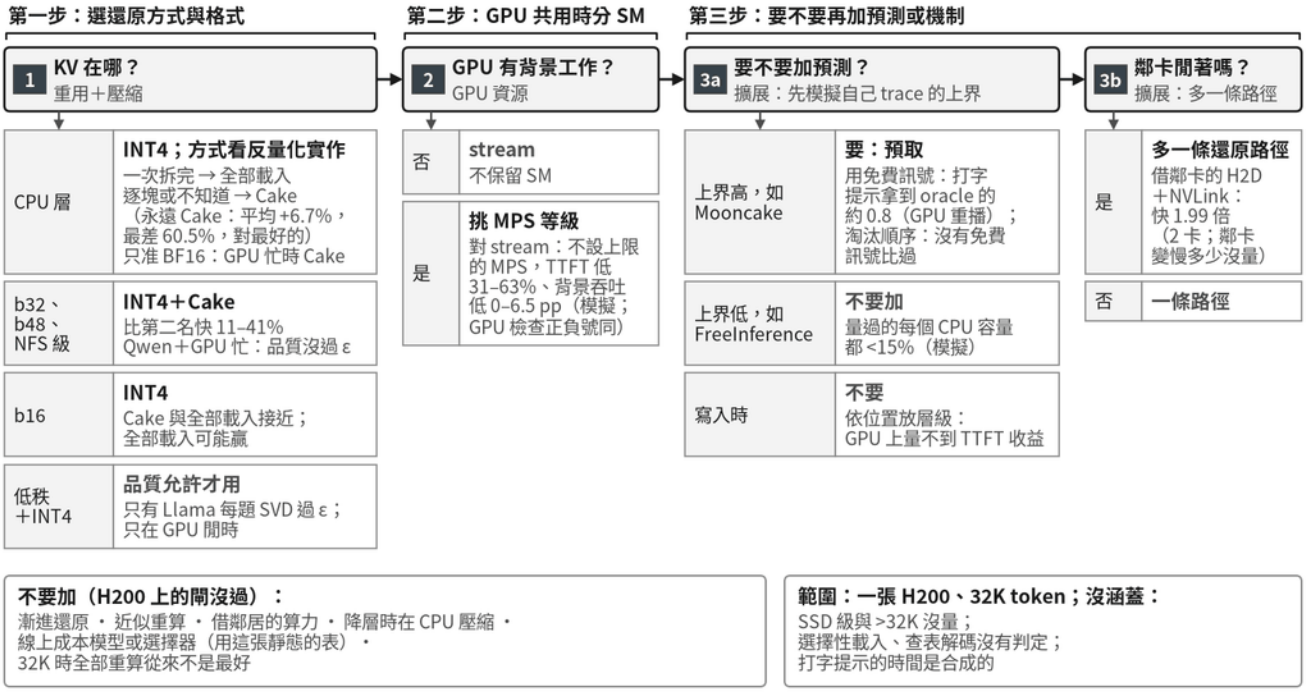


圖 9　送回停放 KV 的決策流程：先查表選送回方式與格式，再分 GPU，最後才考慮預測與多一條路徑。

怎麼讀：由左到右依序回答問題，每一欄的方框是在各種情況下的建議，底下兩框列出不值得做的與本文沒涵蓋的。
代表什麼：照這個順序，大部分情況只要查表與分 GPU；只有在自己的 trace 上量到上界夠高時，才需要預測。

1. **選送回方式｜KV 停在 CPU 記憶體**：先看 INT4 的解壓實作。一次拆完 → INT4 全部載入；逐塊解壓 → Cake；不確定 → 一律 Cake（成本模型的測試格平均多 6.7%，但一次拆完時最多慢到 2.18 倍）[M1、M3-f]。

2. **選送回方式｜KV 停在 b32、b48 或 NFS 級**：INT4＋Cake [M1]。b16 上 Cake 與全部載入差不到 10.4%，選哪個都行；SSD 級沒量，照慢層用 Cake 是外推 [M1]。
3. **選格式｜寫入時能付 SVD 的成本、模型是 Llama**：低秩＋INT4 在 GPU 閒時更快；Qwen 不要用 [M1、M2]。
4. **分 GPU｜要和別的工作共用**：先決定別的工作最少要保有多少處理量，再在自己的 GPU 上試兩三個 MPS 等級；模擬的絕對值不可信 [S3、M4]。
5. **加預測｜**先在自己的 trace 上量 oracle 上界。低於 5% 就不蓋；5–15% 先試打字提示這類免費訊號，不蓋預測器；≥15% 時預取先用打字提示，淘汰順序目前沒有免費訊號 [B3-T5、B9、C2-GPU、B1-ORD]。
6. **加路徑｜有閒置的鄰卡**：多一條還原路徑值得試（只量 2 卡、鄰卡受的影響沒量）；漸進還原、近似重算、借鄰居的算力在我們的門檻下不值得 [G-M06、G-A0、G-M12、G-M08]。

10 限制

結論只在這個範圍內成立：一張 H200、兩個 7–8B 模型、32K 的地圖、限速模擬的慢層，以及經縮小 GPU 重播檢查、但不含排隊的預測模擬。 每條限制後面寫它讓結論往哪邊偏。

限制	對結論的偏向	證據
單卡 H200、7–8B，不做 TP；多一條路徑只量 2 張卡	方向不明；大模型的 KV 更大，送回更貴	§0、G-M06
地圖只到 32K；SSD 級沒有還原格子	SSD 級與更長 context 的最快做法沒有根據	M1
SSD 級與 NFS 級是限速模擬，常數來自 MI300X	真實裝置的干擾與延遲沒算，可能讓慢層看起來較穩	§0
地圖的「忙」是合成的固定負載；背景翻轉只測 CPU 與 b32	b16、b48、NFS 級的穩定性沒有根據	M1、M3-b
b16、b48 用逐塊解壓	這兩層的 INT4 時間偏慢，b16 的平手可能改變	M1
Qwen 的 INT4 在 Cake＋GPU 忙讀法沒過 ε；低秩只有 Llama 每題 SVD 過	Qwen 慢層忙格的建議有品質風險	M2
預測的底接近 §4 的表在逐塊解壓下的答案，沒有 MPS	方向沒有量過	§0、B9
預測的 TTFT 不含排隊；含排隊時預取的上界從 17.9／16.2% 降到 8.1／4.7%	上界偏高	B6
寫入飽和的格讓還沒寫完的資料就能讀	這些格偏樂觀；主結果已排除	§0、B9
單一決策 oracle；聯合 oracle 沒量	可能低估同時預知多件事的上界	§0
打字提示的提前量是合成的	方向不明；真實按鍵時間沒量	§0
FreeInference 只用前 6 週，「知道誰不回來」另用一週；Mooncake 兩條 trace 各一小時，而且有大量相同的請求；FreeInference 的 512 token 換算是近似	trace 的代表性有限；兩條 Mooncake 不算兩份獨立證據	B3-X5、§0
$f(i)$ 只量到約 40K token；FreeInference 前 6 週超過 128K 的比例沒算	長請求的重算成本靠外插	§0
同節點別人的 job 照登記保留；同 NUMA 平行量時間沒驗證	拿掉後判定不變	R-T1G、E4
預測型系統只讀了摘要	文獻的對照組要做 citation audit	無

11 相關研究

前作各自改進一個方向，比較的對象也各不相同；本文把這些選擇放在同一張 H200 上、同一套事先登記的門檻下，依「選組合 → 分 GPU → 加東西」的順序比較。

分層 KV 系統。vLLM 的 offloading 與 tiering、LMCache、Mooncake 與 CachedAttention 都把 KV 移到 CPU 或儲存層，回來時再送回〔文件、ATC'24〕。它們提供機制與預設值，沒有回答哪個組合在哪種條件下最好。本文 §4 量的就是這些機制上方的選擇。

算與載的還原。Cake 讓 GPU 從前段重算、同時從後段載入〔內文〕；HCache 依離線 profile 決定每層用 hidden state 投影、傳 KV 或重算〔部分〕；CacheFlow 用動態規劃（dynamic programming，DP）安排多個 session 的還原〔摘要；先前讀過內文〕；Cascade 用延遲預算決定搬多少、其餘重算〔內文〕；py-kvcache 與 *When the KV Cache Is Slower Than the GPU* 用離線量到的成本決定載或算〔內文〕。它們假設成本可以離線量好，或算與載互不干擾。本文把 Cake 放進格式、解壓實作與 GPU 分配的組合裡比較，並量到在測過的條件下表不翻轉。

壓縮。TensorRT-LLM 只在 host 或磁碟上把 KV 存成 NVFP4〔文件〕；KVServe 依頻寬與 SLO 挑壓縮組合〔內文〕；CacheGen 壓縮 KV 再串流〔內文〕；ShadowKV 與 xKV 用低秩分解〔部分〕。它們多半只看位元組或頻寬。本文把格式和送回方式一起比，量到 INT4 在慢層要配 Cake、在 CPU 層要看解壓實作，以及低秩的寫入成本。

選擇性讀取（聚焦）。ReadKV 與 Fathom 讓 decode 時的 query 決定每塊讀多細〔內文〕；Janus 用原生稀疏模型的索引預測要從 SSD 讀哪些塊〔內文〕；IMPRESS 依新請求挑塊〔部分〕。它們多在 decode，query 是精確的。本文量的是回來那一刻、用新問題決定先讀粗版再精修的漸進還原，結果沒過門檻。

部分重算的重用。CacheBlend 等工作重用已存的 KV、只補算一部分〔未讀內文〕。本文的近似（稀疏）重算屬於這一類，在 32K 回來請求上品質掉太多。

預測型管理。PBKV、SAGA、CacheScout、CacheWise、Continuum、KVFlow 預測下一步呼叫、回來時間或存活時間，報告大的收益〔摘要〕；*Ask the Tool* 用工具回報的進度當免費訊號〔摘要〕；SARIA 與 Gandalf 在寫入時決定准不准進 SSD，目標是寫入量與壽命〔摘要〕。從摘要看，它們的對照組不是分層儲存＋Cake。本文在固定、不預測的底上量預測的 oracle 上界，發現它依 trace 而定。

工作負載特性與極限研究。Liu 等人在 FreeInference 上比較 14 種淘汰策略，發現花俏的策略贏不了 LRU、但離 Belady 遠，miss 的成本只算重算〔內文〕；*Where Should the KV Cache Live?* 用合成 workload 說預取 oracle 也贏不了不預取〔內文〕；*KVCache Cache in the Wild* 描述 trace 的重用特性〔摘要〕。本文的 miss 是從慢層用 Cake 送回、指標是 TTFT，並經 GPU 縮小重播檢查。

GPU 分享與 SM 分割。Tutti 把 SSD 的 I/O kernel 放進獨立的 green context（CUDA 把 SM 分給不同工作的機制）〔部分〕；Bidaw 把 KV 轉換放到低優先權 stream〔內文〕；vLLM 用固定 12 個 SM 的 kernel 搬小塊 KV〔PR〕；NanoFlow、Bullet 等在同一張 GPU 上分 SM 給 prefill 與 decode〔部分〕。查過的 12 個 GPU 共用系統都沒有把 KV 還原放進 SM 分配。本文 §6 量到：改用 MPS 讓送回和別的工作同時跑，回答快很多；替送回保留得越多，別的工作越慢。

12 結論

使用者回來時，停放的 KV 該怎麼送回 GPU，在我們量的 H200 範圍內可以依三步決定。先選送回方式與格式：慢層（b32、b48、NFS 級）用 INT4＋Cake；CPU 記憶體上先看 INT4 的解壓實作，一次拆完就整段載入，否則用 Cake；全部重算從來不是最好，而且這張表在測過的條件下不翻轉，所以查表就夠 [M1、M3]。再分 GPU：要和別的工作共用時，改用 MPS 讓兩者同時跑，在模擬裡別的工作的處理量只少 0–6.5 個百分點；若再限制別的工作、替送回保留更多 GPU，回答會更快，但別的工作會更慢，確切的數字要在自己的 GPU 上量 [S2、S3、M4]。最後才考慮加東西：寫入時的預知不必做；回來時的預測在 FreeInference 前 6 週、測過的 CPU 容量上連完美預知都快不到 15%，在 Mooncake 上預取可用打字提示、淘汰順序還沒有免費訊號；其他機制裡只有借閒置鄰卡多一條還原路徑過了門檻 [A2、B3-T5、B9、C2-GPU、G-M06]。〔判讀〕有效的做法都在減少要搬的位元組、讓搬與算重疊，或多開一條搬的路（INT4、Cake、多一條路徑）；預測只在上界高的 trace 上值得，而且預取用打字提示就拿到大部分。

未來工作（每項對應 §10 的一條限制，都還沒有結果）

- **真實的慢層與 SSD 級的地圖**：在真的 NVMe SSD 與網路儲存上量送回時間，補上 SSD 級。
- **更長的 context 與實作**：把地圖延伸到 64K 與 128K；b16、b48 改用一次拆完的實作重量。
- **真實的負載與品質**：用像真實服務的背景重量 b16、b48、NFS 級的穩定性；為 Qwen 的 INT4＋Cake 找過 ε 的設定。
- **端到端服務**：在有 batching 與排隊的服務系統裡量回來請求的 TTFT，檢查照表選擇與 MPS 的效果。
- **預測**：用更多真實 trace 找出能分開 FreeInference 與 Mooncake、設計者量得到的特徵（例如兩者同容量下的命中率）；用有真實按鍵時間的 trace 驗證打字提示；在 §4 的表上逐格重量預測的增量；量聯合 oracle；為淘汰順序找一個免費訊號。
- **更大的模型與多卡**：在多卡上量多一條還原路徑的收益與對鄰卡的影響。

對照表（內部用，投稿版刪除）

本文的名稱	內部代號	EVIDENCE 列
組合地圖；穩定性；MPS 曲線；四個方向	M1；M3；M4；M5	M1–M5
一次拆完；逐塊解壓	A5 的 S4；D2 的 K0（G5b、C0 用 S0）	M1 注意、M3-g、註 14
每層最佳還原規則；線上選擇器；成本模型	P0-1；G5b；C0	A5、M3-a；E3、M3-e；M3-f
最好的 MPS 等級固定；背景下不翻轉；SM 份額	GB0-STATIC；GB0-FLIP；GB0-SHARE	M3-c、M3-b、M3-d、S1
MPS 端到端；GPU 檢查	E2E-C（K2＝stream＋INT4＋打字提示）；MPS-GPU	S2；S3、M4
四種回來時上界	X1：O-PF、O-ORD（O-BEL、O-BELS）、O-DROP、O-NOW	B1、B1-ORD、B2、B3-X1、B3-T5、B8、B9
知道誰不回來	X5	B3-X5、B5
打字提示預取；提前量	HINT-PF；X3	C1、C2、C2-GPU
GPU 縮小重播	P02（T1G 重量）	B6
命中率差距對 TTFT 差距；FreeInference 小 CPU	T4；T5	B3-T4；B3-T5
更強的還原規劃器	STAIR（T3）	R-T3
寫入時放置；延後決定	S5；S4+	A1''、A1'、A2、A3、A4、A4'
漸進還原；打字時挑塊；依注意力頭還原	A0／A0-C；G-M04；M05	G-A0、M5-聚焦
近似重算；借鄰居的算力；多一條路徑	G-M12；G-M08；G-M06	G-M12、G-M08、G-M06
CPU 上壓縮；VQ 解碼	W0；G-M11	M5-壓縮-d、M5-壓縮-c
CPU 層用 H200 常數；截尾	T1；H3600	§0、R-T1；B2、B9
判定字：值得做／可能／死路	有看頭／可能／死路	§0 門檻

數字出處（內部用）

- 內文與表格的數字都來自 `EVIDENCE.md`；證據欄或方括號是它的列號，[§0] 是設定表。版本照它的規則：模擬用 H200 的 CPU 層常數（T1），GPU 實測用 T1G，更強規劃器（T3）只當敏感度，只用 H200。
- 只出自圖 sidecar 的數字：表 6 的 0.22% 與 0.17%（`figs/fig3_scorecard.sources.json`）；§4 與 §9 的 1.30 倍與 2.18 倍（`figs/fig10_map.sources.json` 的 ratio）。「6 組裡只有 1 組 B ≥0.95」是從 [M4] 的 stream 欄數的。
- §3.2 說預測模擬用逐塊解壓，出自 `code/p3x/x1_sim.py` `constants()` 的反量化常數（AUDIT_v4 #1 核對）。
- 本稿沒有跑新實驗；查核是 self-review，不是 cross-model review。